

Chains Instead of Slots

Week 6 — Singly Linked Lists: Nodes, Links, and Pointer Surgery

Dr. Auqib Hamid Lone

PCC CS-301: Data Structures
GCET, Ganderbal

Week 6

Today's Four Questions

1. Why does inserting one name into a full array cost n moves?
2. What does a node carry that an array slot does not?
3. Which linked shape fits which job — singly, doubly, or circular?
4. How do three pointers turn a chain around without copying it?

Plan

Price the array's weakness. Build the node. Tour the family. Operate: insert, delete, search, display — then reverse.

Handoff: What Week 5 Finished

- ▶ Week 5 gave us the queue contract (enqueue at rear, dequeue from front), the circular fix — $\text{rear} = (\text{rear} + 1) \bmod n$ with the count convention (Lab P5) — dequeues, and array-based priority queues (Lab P6), then closed Unit I for Class Test 1.
- ▶ Every structure so far lives in *numbered slots*: an array owns its elements by position, and position is both its power and its tax.
- ▶ Its closing promise: “what changes when nodes carry their own links instead of sitting in numbered slots.”

This week's job

Free the elements from their slots: each one carries the address of the next. Price what we gain (cheap insert/delete) against what we lose (random access).

The Calculator Needs a Symbol Table

- ▶ Weeks 3–5 built an expression processor: tokenize, check brackets, convert, evaluate, buffer jobs in arrival order. One program learning new tricks each week.
- ▶ New deficiency: evaluating $(a+b)*c$ needs the *values* of a , b , c — identifiers arrive at unpredictable times, in unpredictable numbers. A fixed array demands we guess the capacity up front.
- ▶ The same need appears everywhere: OS free-lists of memory blocks, browser back-forward chains, music playlists, and (Week 7) polynomials with missing terms.

The pattern (design principle)

Every structure in this course answers a deficiency the previous one just exhibited. The array's deficiency is fixed capacity plus shifting — the linked list is the answer.

Where This Week Sits

- ▶ Week 1 gave us `malloc/free` and `NULL`; Week 2 the pricing habit (operation + bound + case). Every claim today repeats that shape.
- ▶ Week 7 re-implements the stack and the queue on top of *this* week's chain, adds the backward link (`p->prev`), and spends the polynomial application; Week 10 replays the same symbol table as a hash table — $O(n)$ as a list vs. $O(1)$ expected hashed.
- ▶ New symbols today: `head` (first-node pointer, `NULL` when empty) and `p->next` (the successor link).

The arc again: state the contract, implement it once, analyse what it costs.

First: price the slot tax that makes chains worth building.

Section 1

Why Chains

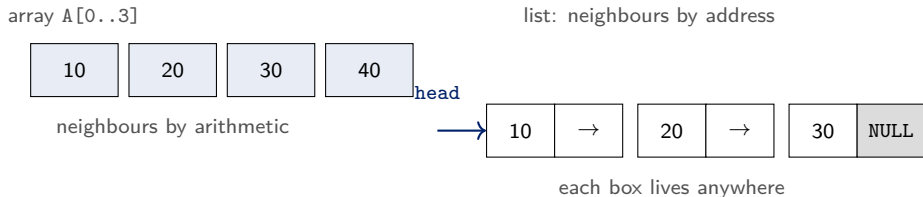
The Slot Tax: Insert Into a Full Array

- ▶ An array of n integers holds $A[0..n-1]$ back to back. Insert x at position k and every element from k to $n-1$ must shift one slot right — up to n moves, $O(n)$ worst case.
- ▶ Delete at k and the same crowd shifts left. Growing past capacity means `malloc` a bigger block, copy all n , free the old one.
- ▶ **Position is the tax:** the array finds any element in $O(1)$ worst case by arithmetic, but pays linear time whenever the lineup itself changes.

The question that buys the list

What if each element carried the address of its successor, so the lineup could change by rewriting one address instead of moving n elements?

Array Slots vs. Linked Nodes, Drawn



- Top: neighbours sit side by side, found by index arithmetic. Bottom: neighbours sit *anywhere* in the heap, found by following arrows from head to NULL.

The Node: Data Plus One Address

Definition (Sedgewick Sec. 1.3, p. 142; Karumanchi Ch. 3, pp.74–162 PDF) [,]

A **linked list** is either empty ($\text{head} = \text{NULL}$) or a node holding data plus a reference to the rest of the list. In C:

```
struct node { int data; struct node *next; };
```

- ▶ Each node is separately malloc'd — Week 1's allocation trace, one box per element, scattered across the heap (stack grows down, heap up, same orientation as Week 1).
- ▶ head is the *only* entry point: lose it and the whole chain leaks. $p \rightarrow \text{next}$ of the last node is NULL.

Worked Example: Build 10–20–30 by Hand

step	heap after the step
head = NULL	empty: head points nowhere
insert 10 at front	head $\rightarrow$ [10 NULL]
insert 20 at front	head $\rightarrow$ [20 .] $\rightarrow$ [10 NULL]
insert 30 at end	head $\rightarrow$ 20 $\rightarrow$ 10 $\rightarrow$ 30 $\rightarrow$ NULL

- ▶ Read the arrows, not the addresses: the three boxes may sit at any heap locations — order lives in the links, never in the layout.
- ▶ Every arrow written is one pointer assignment; every box is one `malloc` that a matching `free` must one day reclaim.

Arrays vs. Linked Lists: The Honest Bill

operation	array	singly linked list
access i -th element	$O(1)$ worst case	$O(n)$ worst case (walk i links)
insert/delete at front	$O(n)$ worst case (shift)	$O(1)$ worst case
insert/delete at end	$O(1)^*$ amortized	$O(n)$ worst case (no tail link)
memory	fixed, contiguous	grows one node at a time

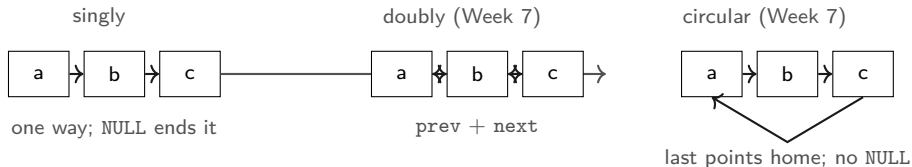
- ▶ *Append is $O(1)$ amortized only with spare capacity; a full array pays the copy-everything reallocation.
- ▶ No winner — only workloads: many positional reads $\rightarrow$ array; many front insertions and deletions $\rightarrow$ list. State the workload, then choose.

The chain is priced. Now meet the whole family.

Section 2

Types and Applications

Three Shapes of Chain



- **Singly** (today): one link each, cheapest per node. **Doubly**: backward walks, twice the link maintenance. **Circular**: round-robin buffers and schedulers.

Applications: Where Chains Earn Their Keep

The rule of thumb (Karumanchi Ch. 3, pp.74–162 PDF; standard treatment) []

Reach for a list when the **number of items is unpredictable** and **insertions and deletions land anywhere** — the two places arrays charge the slot tax.

- ▶ **Symbol table** (our calculator): identifiers come and go; lookup walks the chain — $O(n)$ worst case, the bill Week 10's hash table will cut to $O(1)$ expected.
- ▶ **OS memory management**: free blocks linked as they are freed and re-split on demand — no fixed table of blocks.
- ▶ **Polynomials** (Week 7 preview): $5x^{100} + 3x^2 + 1$ stores three nodes, not 101 array slots mostly holding zero.

Check Your Prediction

A text editor holds 1 000 lines. The user inserts one line at the top, 500 times.
Array or singly linked list?

A: array (indexing wins) B: linked list C: identical either way

1. A — indexing only matters if we jump to line numbers; here the workload is all front insertions.
2. B — each insert rewrites one next pointer: $O(1)$ worst case, where the array shifts up to n lines every time.
3. C — the table two sections back says otherwise; price the workload, not the structure.

Correct answer: B. Reads are sequential either way; the insertions decide.

The family introduced. Now the surgery: five operations, one link at a time.

Section 3

Insert, Delete, Search, Display

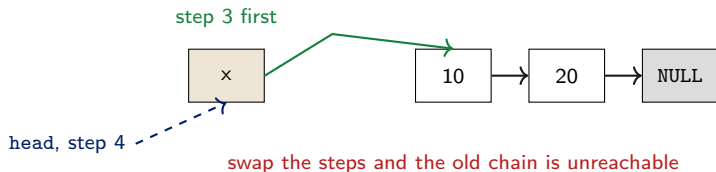
Insert at the Front: Two Assignments

Pointer surgery (Karumanchi Ch. 3, pp.74–162 PDF)

```
newNode = malloc(sizeof(struct node)) // 1. fresh box
newNode->data = x                      // 2. fill it
newNode->next = head                   // 3. point at old chain FIRST
head = newNode                        // 4. THEN swing head over
```

- ▶ Order is the whole lesson: step 4 before step 3 orphans the chain — head's old value is gone with no copy anywhere.
- ▶ Cost: two pointer writes, no walk — $O(1)$ worst case. Always check malloc for NULL before dereferencing.

Insert at the Front, Drawn



- Trace it with a finger: new box points at the old first node, *then* head moves. Either arrow alone is a broken list.

Insert at a Position and at the End

The walk-then-splice pattern

```
p = head; for (i = 0; i < pos-1; i++) p = p->next  
newNode->next = p->next; p->next = newNode
```

- ▶ Walk to the predecessor ($O(n)$ worst case), then the same two writes as front-insert in the same order.
- ▶ End-insert is position n : walk all n links — $O(n)$ worst case without a tail pointer. Guard: inserting into an empty list sets head directly; walking past NULL is a bug, not an edge case.

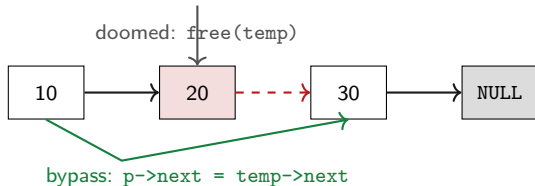
Delete: Find the Predecessor, Bypass, Free

Delete first occurrence of x

```
if head->data == x: temp = head; head = head->next
else: find p with p->next->data == x
      temp = p->next; p->next = temp->next
free(temp)
```

- ▶ Three acts: **bypass** (rewrite one link), **reclaim** (free exactly once), **never touch** temp after the free — that is Week 1's dangling pointer by name.
- ▶ Search-then-delete costs the search: $O(n)$ worst case; the bypass itself is $O(1)$ worst case once p is in hand.

Delete the Middle, Drawn



- ▶ Deleting 20: node 10's link jumps over it straight to 30 (solid green), then 20's box is freed. Skip the free and the node still sits in the heap with nobody pointing at it — Week 1's leak, drawn.

Search and Display: The Two Walks

Both are one loop over $p = p \rightarrow \text{next}$

```
search(x):  p = head; while (p != NULL && p->data != x) p = p->next  
            return (p != NULL)    // found or walked off the end  
display():  p = head; while (p != NULL)  print p->data; p = p->next
```

- ▶ Search is $O(n)$ worst case (the miss walks everything) and $O(1)$ best case (head holds x) — name the case, per the Week 2 habit.
- ▶ Display never modifies: the traversal pointer p moves while head stays put. Moving head itself loses the list.

Four operations banked. Now the classic: turn the chain around in place.

Section 4

Reverse, Costs, Close

Reverse: Motivation Before Mechanism

- ▶ The playlist just queued 500 songs oldest-first; the user taps “play newest first.” Copying all 500 into a second list doubles the memory for a rearrangement.
- ▶ The chain already holds every link we need — each arrow just points the wrong way. Reversal re-points n arrows using three pointers and $O(1)$ extra space.
- ▶ Socratic pause: with only `prev`, `curr`, and `next`, which arrow must be saved *before* overwriting `curr->next` — and why?

The answer (check your instinct)

`next = curr->next` first — overwriting `curr->next` destroys the only path to the rest of the chain.

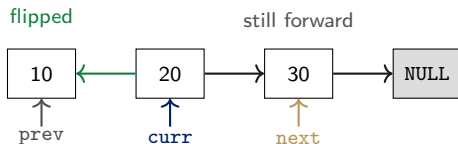
Iterative Reverse: Three Pointers, One Pass

The loop (Karumanchi Ch. 3, pp.74–162 PDF)

```
prev = NULL; curr = head
while (curr != NULL):
    next = curr->next    // save the rest of the chain
    curr->next = prev    // flip this arrow
    prev = curr          // advance both walkers
    curr = next
head = prev
```

- ▶ Each node visited exactly once: $T(n) = n$ link flips — $O(n)$ worst case, $O(1)$ auxiliary space (three pointers, however long the list).
- ▶ Trace $10 \rightarrow 20 \rightarrow 30$ once on paper before the lab: after the second iteration the chain reads $20 \rightarrow 10 \rightarrow \text{NULL}$ with curr parked at 30.

Reverse Mid-Pass, Drawn



- Snapshot after one flip: 10's arrow points back at NULL, prev sits on 10, curr on 20, next saved 30. The frontier between flipped and forward walks right until `curr = NULL`.

Singly Linked List: The Full Price List

operation	bound	note
insert at front	$O(1)$ worst case	two writes; order matters
insert at end / position	$O(n)$ worst case	the walk dominates
delete (value known by walk)	$O(n)$ worst case	bypass itself $O(1)$
search	$O(n)$ worst / $O(1)$ best	miss walks everything
display	$\Theta(n)$	must visit all n
iterative reverse	$O(n)$ worst, $O(1)$ space	one pass, three pointers

- The house rule, Week 2 style: every bound above names its operation and its case. “Insert is $O(1)$ ” without saying *where* is half an answer.

Three Classic Mistakes

- ▶ **Rewriting head before saving it** — in front-insert (step order) and in reverse (unsaved next): the chain's only entry address is gone, and the heap still bills you for every orphaned node.
- ▶ **Walking off the end** — `p = p->next` past NULL dereferences nothing: check `p != NULL` in every loop condition, and test empty and single-node lists first.
- ▶ **Freeing and still pointing** — using `temp` after `free(temp)`, or forgetting the free entirely: Week 1's dangling pointer and leak, now wearing list clothing.

How to revise

Re-trace the insert, delete, and reverse snapshots with a covered answer column — the exam is traces, not definitions.

Week 6 Summary

- ▶ **Chain over slots:** unpredictable size plus anywhere-edits beat $O(1)$ indexing — that is when the list wins; otherwise the array keeps its crown.
- ▶ **Node:** `struct node { int data; struct node *next; };` with head as sole entry and NULL as terminator; singly, doubly, and circular shapes for different jobs.
- ▶ **Five operations:** front-insert $O(1)$ worst case; end/position insert, delete, and search $O(n)$ worst case; display $\Theta(n)$; three-pointer reverse $O(n)$ worst case in $O(1)$ space.

Next week

The backward link (`p->prev`), circular lists, the stack and queue reborn on chains — and polynomials that only store what is nonzero.

Before You Go: Predict

Three one-line predictions

Answer from the algorithms, then check in the lab.

1. List `head → 5 → 15 → NULL`. Insert 10 at the front: write the two assignments in the safe order.
2. Delete 15 above: which link is rewritten, and what is freed?
3. Reverse `head → 1 → 2 → 3 → NULL`: after the loop, what does `head` point at, and what is the cost?

Answers: `newNode->next = head`, then `head = newNode`. Rewrite 5's next to NULL; free 15's node. `head → 3 → 2 → 1 → NULL`; $O(n)$ worst case, $O(1)$ space.

References I



Narasimha Karumanchi.

Data Structures And Algorithms Made Easy.

CareerMonk Publications, 2017.

This calibre-generated edition carries no printed folio numbers; page numbers cited in CS301 material are PDF page indices. NOT a source for Week 1's C-specific pointer/malloc content — see this course's supporting-books index for the coverage check.



Robert Sedgewick and Kevin Wayne.

Algorithms.

Addison-Wesley, 4th edition, 2011.